INSTITUT D'ENSEIGNEMENT SUPÉRIEUR DE RUHENGERI

Accredited by Ministerial Order N° 005/2010/Mineduc of 16 June 2010

FACULTY OF SCIENCES AND INFORMATION TECHNOLOGY

COMPUTER SCIENCE DEPARTMENT

OPTION: INDUSTRIAL INFORMATION TECHNOLOGY (IIT)

LEVEL III

MyTurn: Design and Implementation of an IoT-Based Queue Management and Service-Desk Status System

A project report presented in partial fulfilment of the requirements for the award of a Bachelor's degree of Science with Honours in Computer Science, option of Industrial Information Technology, and for the assessment requirements of the module Internet of Things (IoT).

By:

1. NIYODUHORANA Leah Reg. No: 25/27754

2. DUKUNDIMANA Odilla Reg. No: 25/26987

3. Isaac MUSHIMIYIMANA Reg. No: 24/25783

Supervisor: ……

Course lecturer: Joselyne Byukusenge

Musanze, October 2026

Scientia et Lux

B.P. 155
Ruhengeri
Rwanda

T : +250 788 90 30 30
: +250 788 90 30 32
E : info@ines.ac.rw

W : www.ines.ac.rw

DECLARATION

We, NIYODUHORANA Leah (25/27754), DUKUNDIMANA Odilla (25/26987) and Isaac MUSHIMIYIMANA (24/25783), do hereby declare that the work presented in this report is our own contribution, to the best of our knowledge. The same work has never been submitted to any other University or Institution. We therefore declare that this work is our own for the partial fulfilment of the award of a Bachelor's degree with Honours in Computer Science, option of Industrial Information Technology, at INES-Ruhengeri.

Candidates' names: NIYODUHORANA Leah (25/27754), DUKUNDIMANA Odilla (25/26987) and Isaac MUSHIMIYIMANA (24/25783)

Candidates' signature:

……

Submission date: 23 October 2026

APPROVAL

This is to certify that this report entitled “MyTurn: Design and Implementation of an IoT-Based Queue Management and Service-Desk Status System” is an original study conducted by NIYODUHORANA Leah (25/27754), DUKUNDIMANA Odilla (25/26987) and Isaac MUSHIMIYIMANA (24/25783) under my supervision and guidance.

Supervisor’s name:

……

Supervisor’s signature:

……

Date: ……

DEDICATION

To our parents, who made our studies possible,

to our families, who waited while we worked,

and to every visitor who has stood at a service desk without knowing when the turn would come.

ACKNOWLEDGEMENT

We thank God for the health and the time to carry out this work.

We are grateful to our course lecturer, Mrs Joselyne Byukusenge, who set the requirements for this project, answered our questions and pushed us to show the parts of the system working rather than describing them.

We thank our supervisor for the guidance given during the design, and the staff of the Department of Computer Science at INES-Ruhengeri for the laboratory space and the equipment we were allowed to use.

We also thank the staff of the service offices who let us watch how visitors are received, and the classmates who agreed to act as visitors during our tests. Their patience made the measurements in this report possible.

Finally, we thank our families for their support and encouragement during the four days of building and testing.

ABSTRACT

Service offices at INES-Ruhengeri receive several visitors at the same time, and where the order is kept by an informal line or by staff calling out names, a visitor cannot tell who is next, which ticket is being served or whether the desk has paused. MyTurn is a low-cost Internet of Things system that answers those three questions for one service desk. The prototype combines an ESP32 microcontroller, an HC-SR04 ultrasonic sensor, an HC-SR501 motion sensor and an RGB status light with a web application that the team hosts itself.

The sensors report whether a visitor is standing at the service position and when someone is walking towards it, the light shows ready, occupied or paused, and a Python Flask API with an SQLite database issues anonymous numbered tickets, stores desk settings and exchanges commands with the device. Visitors join the queue by scanning a QR code, or staff issue a ticket for a visitor without a smartphone, and staff call, complete, skip or cancel tickets from a dashboard behind a password. The system was built in four short iterations and tested with labelled occupancy and approach trials, twenty test tickets and timed checks of the notification and control paths.

The tests reached **[number]** correct classifications out of 20 labelled occupancy trials, **[number]** of the ten approaches were reported within two seconds, all twenty test tickets kept their order, and a pause command reached the light in **[number]** seconds on average. The prototype shows that one desk can be organised with about RWF 46,200 of hardware and free software, and that a self-hosted API can carry both device telemetry and the queue logic without an IoT platform.

Keywords: Internet of Things, queue management, ESP32, HC-SR04, HC-SR501, Flask, SQLite, remote device control.

TABLE OF CONTENTS

DECLARATION i
APPROVAL ii
DEDICATION iii
ACKNOWLEDGEMENT iv
ABSTRACT v
LIST OF ABBREVIATIONS AND ACRONYMS x
CHAPTER I: GENERAL INTRODUCTION 1
1.1 Background 1
1.2 Problem statement 2
1.3 Objectives of the study 3
1.3.1 General objective 3
1.3.2 Specific objectives 3
1.4 Research questions 3
1.5 Choice of the study 4
1.6 Delimitation of the study 4
1.7 Significance of the study 4
1.8 Research methodology 5
1.9 Organization of the study 5
CHAPTER II: LITERATURE REVIEW 6
2.1 Introduction 6
2.2 Conceptual review 6
2.2.1 Turn-taking and queue management 6
2.2.2 The Internet of Things in service delivery 6
2.2.3 Microcontrollers and sensors for presence detection 7
2.2.4 Communication between devices and servers 8
2.2.5 IoT platforms and self-hosted APIs 9
2.2.6 Agile development for short projects 9
2.3 Related works and gaps 9
CHAPTER III: RESEARCH METHODOLOGY 11
3.1 Introduction 11
3.2 Research design 11
3.3 Development methodology 11
3.4 Requirements gathering 11
3.5 Tools and technologies 12

3.6 Implementation steps 13
3.7 Testing and validation approach 13
3.8 Data collection and analysis 14
3.9 Ethical considerations 15
CHAPTER IV: SYSTEM DESIGN, IMPLEMENTATION AND RESULTS 16
4.1 Introduction 16
4.2 Analysis of the current system 16
4.3 System requirements 17
4.3.1 Functional requirements 17
4.3.2 Non-functional requirements 17
4.4 System design 18
4.4.1 System architecture 18
4.4.2 Use case diagram 19
4.4.3 Data flow 19
4.4.4 Database design 20
4.4.5 Hardware design 21
4.5 Implementation 22
4.5.1 Device firmware 22
4.5.2 Application programming interface 23
4.5.3 Browser interfaces 24
4.6 Testing and validation 27
4.7 Results and discussion 28
4.7.1 Results 28
4.7.2 Discussion of the results 29
CHAPTER V: CONCLUSION AND RECOMMENDATIONS 31
5.1 Introduction 31
5.2 Conclusion 31
5.3 Challenges encountered 32
5.4 Recommendations and future work 33
REFERENCES 35
APPENDICES 37
Appendix A: ESP32 device firmware 37
Appendix B: Flask application routes 40
Appendix C: Database schema 42
Appendix D: Test record sheet 43

LIST OF TABLES

Table 1: Abbreviations and acronyms used in this report x
Table 2: Tools and technologies used in the project 12
Table 3: The four build iterations and the changes they caused 13
Table 4: Acceptance criteria used to test the prototype 14
Table 5: Functional requirements of the MyTurn system 17
Table 6: Non-functional requirements of the MyTurn system 17
Table 7: Tables in the MyTurn database 21
Table 8: Pin connections between the ESP32, the sensors and the status light 22
Table 9: API routes of the MyTurn application 23
Table 10: Results of the acceptance tests 27
Table 11: Occupancy trial record sheet 43
Table 12: Approach and walk-past record sheet 43
Table 13: Timing record sheet 44
Table 14: Ticket order record sheet 44

LIST OF FIGURES

Figure 1: Architecture of the MyTurn system for one service desk 18
Figure 2: Use cases of the MyTurn system and the actors that start them..... 19
Figure 3: End-to-end data flow between the visitor, the staff, the API, the database and the device..... 20
Figure 4: Entity relationship diagram of the MyTurn database..... 20
Figure 5: Wiring of the ESP32, the ultrasonic sensor, the motion sensor and the RGB status light 22
Figure 6: Visitor page on a phone, showing the ticket number, the called number and the desk state 27
Figure 7: Staff dashboard, showing the queue and the call, complete and skip actions 27
Figure 8: The desk during a test, with the sensor above the service position and the status light in view 27
Figure 9: Occupancy trial results: correct and incorrect classifications out of twenty labelled trials..... 28
Figure 10: Delay between a staff action and the result showing on the page and on the light 29

LIST OF ABBREVIATIONS AND ACRONYMS

Table 1: Abbreviations and acronyms used in this report

Abbreviation	Meaning
API	Application Programming Interface
CoAP	Constrained Application Protocol
CSS	Cascading Style Sheets
ERD	Entity Relationship Diagram
FCFS	First Come, First Served
GPIO	General Purpose Input Output
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
INES	Institut d'Enseignement Supérieur de Ruhengeri
IoT	Internet of Things
JSON	JavaScript Object Notation
LED	Light Emitting Diode
MQTT	Message Queuing Telemetry Transport
QR	Quick Response (two-dimensional barcode)
RWF	Rwandan Franc
SQL	Structured Query Language
SQLite	A small file-based SQL database engine
USB	Universal Serial Bus
Wi-Fi	Wireless local area network technology

CHAPTER I: GENERAL INTRODUCTION

1.1 Background

Service offices in higher learning institutions do most of their work in person. A student who needs a transcript, a change of registration or a payment confirmation walks to the office that handles it and waits there. When several students arrive within a short period, the office has to decide who is served first, and that decision is usually made with an informal physical line or by a staff member calling out names from a list.

Neither method tells a visitor much. A student who steps out of the line loses a place that was never written down. A student who arrives at a busy moment cannot tell how many people are ahead or how long the wait will be, and a student waiting in the corridor cannot tell whether the counter is serving someone, whether the staff member has stepped out or whether the counter has closed for the day. Staff carry the extra work of answering the same questions about the order and the waiting time, which interrupts the service itself.

Internet of Things technology can reduce this uncertainty at a low cost. A small microcontroller with built-in Wi-Fi, such as the ESP32, can read a sensor, send the reading to a server and receive instructions in return (Espressif Systems, 2023). An ultrasonic sensor can tell whether a person is standing at one position, and a passive infrared sensor, which reacts to movement rather than to distance, can tell that someone is walking towards the desk before they reach it. A light can show the state of the desk to anyone in the corridor. On the software side, a Python web application with a small database can issue numbered tickets, keep the queue in order and serve pages to any phone browser that has a camera and Wi-Fi (Pallets Projects, 2023; Hipp, 2020).

MyTurn applies those parts to a single service desk. Visitors take an anonymous numbered ticket by scanning a QR code, or a staff member issues a ticket for a visitor who has no smartphone. The server allocates numbers in order, staff call the next ticket from a dashboard, the called number appears on a public screen and on the visitor’s own page, and two sensors with a status light show whether the service

position is occupied, ready or paused, and whether someone is walking towards it. The work was carried out for the module Internet of Things (IoT) in the final level of the Bachelor's degree programme in Computer Science, option of Industrial Information Technology.

1.2 Problem statement

Where service depends on an informal line or on staff calling out names, a visitor cannot tell who is next, which ticket is being served, or whether the desk has paused. Visitors then crowd around the desk, ask the same questions again, disagree about turns and interrupt office work. Visitors without smartphones are in the same position, because any solution has to give them a way into the same queue rather than a separate one.

A light that only shows whether a desk is empty does not solve the problem. An empty chair says nothing about whether staff are available, and it does not identify the next visitor. What the desk needs is an agreed order that everyone can see, a service status that staff control, and a signal showing that a visitor has reached the service position. Those three needs pull in different directions: the order belongs to the software, the status belongs to the staff, and the signal belongs to the device.

The literature on service encounters supports the direction the project takes. How long a wait feels depends on how much the person knows about it, and uncertainty about the order makes a wait feel longer than the same wait explained in advance (Maister, 1985; Bitner, 1990). The project therefore aims at the uncertainty rather than at the waiting time itself.

The team did not measure queue lengths or waiting times at any office. The description above is what the team observed and heard during short visits and conversations, and it is stated as the working assumption of the design, not as the result of a survey.

1.3 Objectives of the study

1.3.1 General objective

To design, implement and demonstrate a low-cost IoT prototype that organises visitors at one service desk and reports ticket calls, desk occupancy, visitor approach and staff-controlled service status through a web application and a physical indicator.

1.3.2 Specific objectives

- To build a ticket system that issues unique sequential numbers on a first-come, first-served basis and lets staff issue a ticket for a visitor who has no smartphone.
- To build a sensor circuit, using an ESP32 with an ultrasonic sensor and a passive infrared sensor, that detects occupancy at the service position and the approach of a visitor, and reports both to the server.
- To write a custom Flask API and SQLite database that receive device readings, store tickets and desk settings, and exchange control commands, without using Blynk, ThingSpeak or a similar IoT platform.
- To provide a public page for ticket and status information and a staff dashboard behind a login, so that staff can call, complete, skip or cancel tickets and pause or resume service.
- To test occupancy detection, approach detection and walk-past rejection, ticket order, notification updates, remote light control, data persistence and connection handling, and to record the results.

1.4 Research questions

1. How can the order of visitors at one service desk be kept and displayed without collecting personal information about them?
2. How accurately can a low-cost ultrasonic sensor detect whether a visitor is standing at the service position, and a passive infrared sensor detect that someone is walking towards it?
3. How can a device receive a control command from a web dashboard and report back whether the command was applied?

4. Which parts of the service workflow can be carried by a self-hosted API and a small database, without handing the data or the control logic to an IoT platform?

1.5 Choice of the study

The topic was chosen for four reasons. First, the problem is familiar to the team: every member has waited at a registration or finance desk without knowing when the turn would come. Second, the module brief requires a prototype that contains a microcontroller, sensors and a data communication path to a server the team controls, and this design needs all three. Third, the parts are affordable: the whole prototype is estimated at RWF 46,200, and the laptop, phone and Wi-Fi access point were already available. Fourth, the result can be built and checked within the week allowed for the assignment, because the hardware is small and the software is a single web application.

The project also gives the team practice in the part of IoT that is easiest to skip in a classroom exercise: the server side. Sending a reading to a public platform takes a few lines, while keeping the queue logic, the device authentication and the control path in one place requires design decisions that this report documents.

1.6 Delimitation of the study

Five limits bound the work. The prototype serves one desk and one queue, so there is no management of several counters and no transfer of visitors between them. Service is first come, first served, and there are no priority or appointment queues. Tickets are anonymous: the system stores a ticket number and a random access token, and never a name, a telephone number, a photograph or a student registration number. Notifications reach an open browser page or the public screen only; there are no SMS, WhatsApp or push messages to a locked phone. Finally, the evaluation uses a small number of labelled trials with team members acting as visitors, so the accuracy figures describe the prototype under test conditions and not a long deployment in a busy office.

1.7 Significance of the study

For visitors, the system replaces a guess with information: the number being served, the number of people ahead and the state of the desk are visible from the corridor and

from a phone. Visitors without smartphones are served from the same queue rather than a separate one.

For staff, the dashboard makes the order of service an explicit decision, and the light takes over the work of telling people when the desk is busy, paused or closed. The occupancy record also shows how long the service position was used, which the office can review later.

For the institution, the prototype shows that a service improvement can be attempted with parts worth about RWF 46,200 and free software, and that the data can stay on a server the institution controls.

For the team, the project brings together hardware and software in one system: a circuit that has to be wired and calibrated, firmware that has to survive a weak network, a database that has to keep a queue in order under two simultaneous requests, and a user interface that has to be understandable without training.

1.8 Research methodology

The work follows an applied, prototype-based design. Requirements were gathered by observation at a service desk, by short conversations with staff and students, and from the module brief. The system was built in four short iterations, one part per iteration, with a review after each: the sensor circuits first, then the server connection and the light control, then the ticket logic and the browser pages, and finally the acceptance tests. Testing mixed labelled occupancy trials, ten approach and walk-past trials, twenty test tickets, timed notification and control checks, and restart tests for persistence. Chapter III describes the method in full.

1.9 Organization of the study

Chapter One introduces the problem, the objectives and the limits of the work. Chapter Two reviews the concepts and the published work behind the design. Chapter Three describes the methodology, the tools and the implementation steps. Chapter Four presents the system design, the implementation and the results of the tests with a discussion of what they show. Chapter Five concludes the report, lists the challenges met and recommends the improvements that a longer project could add. References and appendices close the report.

CHAPTER II: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the ideas the design rests on. It starts with queueing and the service encounter, then follows the Internet of Things from the device to the application, looks at the sensors and communication options available at this price level, compares a public IoT platform with a self-hosted API, and closes with the development approach used for a short project. The last section states where the reviewed work stops and what MyTurn adds.

2.2 Conceptual review

2.2.1 Turn-taking and queue management

A queue is a rule for deciding who is served next. First come, first served is the rule most people expect at a service desk, because it treats every arrival equally and is easy to explain. The rule matters less than whether the people in the queue can see it working. Studies of service encounters show that a customer judges a service partly on the circumstances around the service itself, including the physical setting and the behaviour of the staff (Bitner, 1990), and that an unexplained wait is felt as longer than the same wait that is explained (Maister, 1985).

Electronic queue systems apply that idea by replacing the informal line with a printed or digital ticket and a display. Banks, hospitals and large offices use them to keep an order across several counters. Such products are usually sold with a licence, a monthly fee and a set of counters, and they assume that the institution is willing to hand the queue data to the supplier. A single university desk does not need the scale, but it does need the order.

2.2.2 The Internet of Things in service delivery

The Internet of Things describes physical objects that carry sensors or actuators and are connected to a network, so that a remote system can observe or influence the physical world (Atzori, Iera and Morabito, 2010). Work on the subject usually separates the system into a device layer, a network layer and an application layer, and treats the value of the system as coming from the combination rather than from any

one layer (Gubbi, Buyya, Marusic and Palaniswami, 2013). That division matches MyTurn: the ESP32, the sensors and the light form the device layer, Wi-Fi carries the readings, and the Flask application with its database forms the application layer.

Service delivery is a common setting for small IoT systems, because the objects involved are cheap and the questions are local: is a room in use, has a parcel arrived, is a machine free (Dargie and Poellabauer, 2010). The constraint on such systems is the device. A microcontroller on a desk has kilobytes of memory, a modest radio and no operator, so the messages it sends have to be short and the network has to be treated as unreliable.

2.2.3 Microcontrollers and sensors for presence detection

The ESP32 was chosen as the controller. It is a dual-core microcontroller with Wi-Fi and Bluetooth built in, a set of general-purpose input and output pins, and support for the Arduino environment, which shortens development time (Espressif Systems, 2023). An Arduino Uno with a separate Wi-Fi shield would cost more and use more wiring, and a Raspberry Pi would add a full operating system that this task does not need.

For presence detection the design uses the HC-SR04 ultrasonic module. It measures distance by timing a sound pulse, covers roughly 2 cm to 400 cm, and has a measuring angle of about 15 degrees (Elec Freaks, n.d.). It is inexpensive and widely available, which matters for a prototype. Its limitations are known: soft clothing reflects sound poorly, a person who is standing very still can be missed between measurements, an angled body returns a weaker echo, and the module answers the first reflection it hears, so a nearby wall or a second sensor can create false readings. The design answers these limits by fixing the sensor on one position, by requiring several consistent readings before the state changes, and by keeping the measuring range inside the desk area.

The second sensor is a passive infrared module, the HC-SR501. It senses the change in infrared radiation caused by a moving body and holds its output high for a short time. Because it reacts to movement, a visitor who stands still fades from its view, so it cannot be used to decide whether the position is occupied; it also needs about half a minute to settle after power is applied (Adafruit, n.d.). Its strength is the opposite of

the distance sensor, because it covers a wide cone and notices a person who is still several metres away. The design therefore uses the two modules together: the infrared module watches the approach, and the ultrasonic module decides the occupancy state at the desk.

Other sensors were considered. An infrared break beam needs an emitter and a receiver on opposite sides of the passage and gives a binary result only. A time-of-flight laser module measures distance more precisely but costs several times as much, and a millimetre-wave radar module holds a still person better but is outside the budget of this prototype. The pair that was kept answers its own weaknesses: the distance reading carries the authority on whether someone is at the desk, and the motion reading supplies the warning that the distance sensor alone cannot give.

2.2.4 Communication between devices and servers

Three protocols are commonly used to move data from a constrained device to a server. HTTP follows a request and response pattern over a TCP connection and carries text headers with every message (Fielding and Reschke, 2014). MQTT uses a publish and subscribe model through a broker, which keeps messages small and suits many devices reporting to one point, at the cost of running the broker (OASIS, 2019). CoAP is a lighter, datagram-based protocol with a REST-like style, designed for devices with very little memory (Shelby, Hartke and Bormann, 2014).

The assessment brief for this module accepts MQTT, HTTP or CoAP. MyTurn uses HTTP because the Flask application already speaks it: the same route that records a reading can be tested from a browser, and no broker has to be installed on the server. The cost is a larger header on every message, which is acceptable for one device sending a reading every few seconds.

The direction of the control path needs care. A device behind a router cannot be reached from the internet without port forwarding or a tunnel, so the server cannot push a command to it. MyTurn therefore has the device ask the server for the current setting on a short interval, which is polling. Polling costs more requests than a push connection, and it is used here because it works through the same outbound connection the readings already use.

2.2.5 IoT platforms and self-hosted APIs

Public IoT platforms such as Blynk and ThingSpeak provide a device library, a dashboard and a hosted database within minutes, and they are a reasonable choice when the goal is to see a sensor reading on a graph (Blynk, 2023; MathWorks, n.d.). They also put limits on the arrangement: the data sits in an account on the provider's server, the free tiers restrict the message rate, the history or the number of devices, and the control rules are expressed in the platform's own interface. The module brief for this project excludes such platforms and requires the team to send data to its own server.

Writing the server also has a technical reason. The queue is application logic, not device telemetry. Deciding which ticket is next, refusing two calls at once and closing a paused desk are database transactions with rules attached. Keeping them in the same application as the device routes means there is one source of truth for the state of the desk.

2.2.6 Agile development for short projects

Agile development favours working software, short feedback cycles and responses to change over a fixed plan and heavy documentation (Beck et al., 2001). Scrum gives the idea a specific shape with roles, events and artefacts (Schwaber and Sutherland, 2020). A project of four days cannot carry the full set of ceremonies, so this work uses a light form of the same idea: a fixed minimum feature set agreed at the start, one working part per iteration, a short review at the end of each day, and a decision to change the plan when a test failed.

2.3 Related works and gaps

Three groups of related work were reviewed. The first is commercial and institutional queue management, which solves turn-taking well and is priced and sized for several counters. The second is occupancy sensing, which is studied in smart buildings and usually reports to a building management system rather than to a queue (Dargie and Poellabauer, 2010). The third is classroom and student IoT projects that send readings from a microcontroller to a platform such as Blynk or ThingSpeak and display them on a phone.

Each group covers part of the problem. Queue systems do not sense the desk, occupancy systems do not manage a turn, and platform-based student projects usually stop at showing a reading, so the control path and the server-side rules are never exercised. The team did not find a published low-cost design that combines anonymous ticketing, staff-controlled service status and desk occupancy and approach sensing in one small system.

MyTurn fills that gap in a modest way. It takes the turn logic from the queue systems, the sensing idea from the building literature, and the device and software parts from the IoT literature, then keeps all of them on a server the team controls. The originality claimed here is the combination and the constraints it is built under, not any single part of it.

CHAPTER III: RESEARCH METHODOLOGY

3.1 Introduction

This chapter explains how the project was carried out. It states the research design, the development approach, the way requirements were gathered, the tools and technologies used, the implementation steps followed, the way the prototype was tested, and the ethical rules the team kept during the work.

3.2 Research design

The work is applied research with a prototype as its output. The question it answers is practical: whether a small and affordable device can organise one service desk. Reading alone cannot answer it, so the answer had to be built. The design combines a small amount of qualitative work, which is the observation of how visitors and staff behave at a desk, with quantitative measurement, which is the number of correct occupancy classifications, the time taken by a notification and the number of ticket numbers issued without a clash. Chapter Four reports both kinds of evidence.

3.3 Development methodology

The prototype was built with a light form of Agile development. The full set of Scrum ceremonies is not practical in a project of four days, so the team kept the parts that carry the value in a short cycle: a fixed minimum feature set agreed before coding began, one working part per iteration, a review at the end of each iteration, and a willingness to change the plan when a test failed. The minimum feature set was the five items listed in the specific objectives: numbered tickets, sensor detection, a self-hosted API with a database, a public page with a staff dashboard, and recorded test results. Anything outside that set was treated as an improvement and left out of the first four iterations.

3.4 Requirements gathering

Requirements came from three sources. The first was observation: two members of the team spent a morning at a student service office, watching where visitors stood, how they learned that it was their turn and what they did while waiting. The second was short conversations, six in total, with staff and students who use the desk; the

questions asked were how the order is kept now, what interrupts the work of the office, and what a visitor usually asks. The third was the module brief, which fixes the technical requirements: a microcontroller, a sensor, data communication to a server the team controls, and remote control of a device from a dashboard.

The observation and the conversations were informal and are treated as design input rather than as research data. No questionnaire was distributed and no personal information was recorded, so no requirement in this report is presented as a measured finding about the office. The functional and non-functional requirements that came out of this step are listed in Chapter Four.

3.5 Tools and technologies

Table 2 lists the tools and technologies used, with the part each one plays. All software is free or open source, and the hardware is the only cost in the project.

Table 2: Tools and technologies used in the project

Tool or technology	Part played in the project
ESP32 DevKitC board	Runs the firmware, reads the sensors, drives the status light and exchanges messages with the API over Wi-Fi
Arduino IDE with the ESP32 board package	Writes, uploads and debugs the firmware and shows the serial monitor during calibration
HC-SR04 ultrasonic sensor	Measures the distance from the sensor to the visitor position
HC-SR501 passive infrared sensor	Notices a visitor walking towards the desk, before the distance reading changes
RGB LED with three 220 Ω resistors	Shows the state of the desk (green, red or yellow) from the corridor
1 kΩ and 1.8 kΩ resistors	Divide the 5 V echo pulse of the distance sensor to a voltage the ESP32 input accepts
Breadboard, jumper wires and a USB cable	Builds and rebuilds the circuit without soldering and powers the board
Python 3 with Flask	Serves the API, the visitor page, the public screen and the staff dashboard
SQLite	Stores tickets, ticket events, desk settings, devices and device reports in one file
HTML, CSS and JavaScript	Build the three browser interfaces and poll the API for updates
Git	Keeps a version of the code at the end of each iteration, so a broken change could be undone
Free hosting account or a tunnel to the laptop	Makes the API reachable over HTTPS from the device and from phones during the demonstration
Phone camera and a QR code reader	Joins the queue without typing a web address
Chrome with the developer tools	Checks the pages and inspects the JSON responses while the API is being built

3.6 Implementation steps

The build was divided into four iterations. Table 3 gives the goal of each iteration, what it produced and what the review at the end of the iteration changed. The plan in the project proposal has the same four parts, so this table is also the record of what was actually done.

Table 3: The four build iterations and the changes they caused

Iteration	Goal	Output at the review	What the review changed
1. Circuit and sensor	Wire the ESP32 with the sensor and the LED, and find a distance threshold that separates an empty position from an occupied one	Distance readings on the serial monitor and a green light that responded to a hand	Readings jumped when a target moved slowly, so a median of five readings and a two-second hold were added before the state may change
2. Server link and light control	Build the Flask API with device authentication, reading upload and command polling	The board reported readings and followed a setting chosen in a browser	The board kept an old setting after a network break, so a retry was added and the last seen time of the device was stored and displayed
3. Queue and browser pages	Add ticket issue, the staff dashboard, the login, the public screen and QR access	A complete workflow from taking a ticket to calling and closing it	Two staff windows could call the same next ticket, so the choice was moved into one database transaction and the second call was refused
4. Tests, fixes and report	Run the acceptance tests, fix priority defects and write the report	Measured results, the corrected interface and this document	One labelled trial missed a still visitor, which is recorded as a limitation and covered by the hold rule

3.7 Testing and validation approach

The nine checks below were agreed before the build started, so that success was defined in advance rather than after the results were known. Table 4 lists them. Occupancy was tested with twenty labelled trials, ten with a person standing at the service position and ten with the position empty, and the sensor result was compared with the label. Approach detection was tested with ten walks towards the desk, and walk-past rejection with ten walks past it, to see whether the motion sensor alone would report an occupancy that the distance sensor did not confirm. Ticket order was

tested by issuing twenty tickets, two of them from two browser windows at the same moment, to see whether the numbers stayed unique and in order. Timing was tested by stamping the moment a staff action was sent and the moment the visitor page or the light showed the result. Persistence was tested by restarting the server and the board and checking that the queue was unchanged. Connection handling was tested by switching off the access point for thirty seconds. Access control was tested by sending a staff request without a session and a device request with a wrong key.

Table 4: Acceptance criteria used to test the prototype

No.	Check	What is measured	Pass condition
1	Ticket order	Twenty tickets issued, two of them at the same moment	Every number is unique and the numbers follow the order in which the tickets were requested
2	Occupancy detection	Twenty labelled trials, ten occupied and ten empty	At least 18 of the 20 trials are classified correctly
3	Approach detection	Ten walks towards the desk, timed from the moment the visitor enters the marked zone	At least 9 of the 10 approaches are reported within 2 seconds
4	Walk-past rejection	Ten walks past the desk without a stop	The desk is not reported as occupied in any of the ten trials
5	Call notification	A staff member calls the next ticket	The number appears on the visitor page and the public screen within 5 seconds
6	Remote light control	A staff member pauses and resumes the desk	The light shows yellow and then green within 5 seconds of each action
7	Persistence	The server and the board are restarted	Tickets, ticket events and the desk setting are all still present
8	Connection loss	The access point is switched off for 30 seconds	The board reconnects without help and the queue is unchanged
9	Access control	A staff action without a session and a device request with a wrong key	Both requests are refused and nothing is changed

3.8 Data collection and analysis

Three kinds of data were collected. The first is the classification result of each occupancy trial, recorded on paper together with the label and the measured distance, and the result of each approach and walk-past trial, written beside the moment the visitor entered the marked zone and the state the dashboard showed. The second is the delay between a staff action and the moment the result appeared, taken from timestamps written by the server and read from the database after the run. The third is the record kept by the system itself: every ticket, every state change and every device

reading stored in SQLite, which allows the order and the timing to be re-checked afterwards rather than trusted to memory. Analysis is descriptive. The classification results are counted, the delays are averaged and the range is noted, and the ticket record is inspected for repeated or missing numbers. No statistical test is applied, because the number of trials is small and the purpose of the tests is to show that the prototype works, not to estimate the accuracy of the sensor in general.

3.9 Ethical considerations

The system stores no personal information. A ticket carries a number and a random token, and the token is used only to let one visitor page follow one ticket, so the queue cannot be linked to a student record. The sensor measures distance and does not record images or sound, and no camera is used anywhere in the prototype. The people who took part in the trials were team members and classmates who agreed in advance, and the labelled trials were run in the laboratory rather than in a live office, so no visitor to a real service desk was observed for the purpose of this report. The circuit runs on 5 V from a USB port, which carries no risk of electric shock, and the sensor and the light were mounted so that they do not block a passage. Where the report quotes what staff said during the short conversations, the statements are attributed to no individual and no office is named.

CHAPTER IV: SYSTEM DESIGN, IMPLEMENTATION AND RESULTS

4.1 Introduction

This chapter presents the system that was built. It starts with an analysis of the way the desk works without the system, states the requirements that came out of that analysis, and then describes the design with its diagrams, the database, the API, the device firmware and the browser interfaces. The last two sections report the results of the acceptance tests and discuss what they show.

4.2 Analysis of the current system

At present a visitor walks to the service desk and joins whatever order exists at that moment. In a quiet period the order is simply the sequence of arrivals, and staff serve whoever is standing in front of them. In a busy period the order becomes a line along the corridor, or a list of names on a sheet when the visitors are known to the office. The turn of a visitor is known to the staff and to the people immediately around the desk, and to nobody else.

Four weaknesses follow. A visitor who steps away loses a place that was never written down. A visitor who arrives later cannot tell how many people are ahead. A visitor who is waiting outside the office cannot tell whether the desk is serving, paused or closed, so the same question is asked at the door several times an hour. Staff also have no record of how long the service position was actually used, which would show whether the desk is busy or merely crowded.

The system keeps the parts of the present arrangement that work and replaces the rest. Staff still decide when a visitor is finished and who is called next, because only they can judge that. What changes is that the order, the called number and the state of the desk become visible to everyone, including visitors who are standing outside.

4.3 System requirements

4.3.1 Functional requirements

Table 5 lists what the system must do. The list is the agreed minimum feature set from Chapter Three, written as statements that can be tested one by one.

Table 5: Functional requirements of the MyTurn system

No.	Requirement
FR1	The system shall give every request for a ticket a unique number, in the order in which the requests arrive.
FR2	The system shall allow a staff member to issue a ticket for a visitor who does not use a smartphone.
FR3	The system shall show a visitor the number of the ticket being called and how many tickets are waiting ahead.
FR4	The system shall show the called number, the number of waiting tickets and the state of the desk on a public screen.
FR5	The system shall allow a signed-in staff member to call the next ticket, and shall refuse a second call for the same ticket.
FR6	The system shall allow a signed-in staff member to complete, skip or cancel a ticket.
FR7	The system shall allow a signed-in staff member to pause, resume or close the desk.
FR8	The system shall detect whether a visitor is standing at the service position and record the state with a timestamp.
FR9	The system shall record an approach when the motion sensor detects a visitor walking towards the desk, report it with a timestamp, and treat an approach that is not followed by an occupancy reading as a walk-past.
FR10	The system shall light the status indicator green when the desk is active and the position is free, red when a visitor is at the position, and yellow when the desk is paused or closed.
FR11	The system shall accept occupancy and approach reports only from an authenticated device.
FR12	The system shall let the device collect the current service setting and report the state it has applied.
FR13	The system shall keep tickets, ticket events and the desk setting after a restart of the server or the device.
FR14	The system shall allow a staff member to sign in with a password and to sign out.

4.3.2 Non-functional requirements

Table 6 lists the qualities the system had to keep while meeting the functional requirements.

Table 6: Non-functional requirements of the MyTurn system

No.	Quality	Requirement
NFR1	Usability	A visitor needs no training and no printed instructions beyond the label next to the QR code.

No.	Quality	Requirement
NFR2	Response time	A call, a pause or a resume appears on the browser pages within five seconds.
NFR3	Reliability	The device reconnects after a network loss of at least thirty seconds and no ticket is lost or reordered.
NFR4	Affordability	The whole prototype costs no more than about RWF 46,200, using free software.
NFR5	Security	Staff routes require a signed-in session, device routes require a device key, and the hosted demonstration is served over HTTPS.
NFR6	Privacy	No name, telephone number, photograph or student registration number is stored, and the ticket token is random.
NFR7	Extensibility	The device table keeps one row per board and the desk setting is stored as a row, so a desk column on the ticket table is enough to extend the prototype to several desks.
NFR8	Maintainability	One web application, a documented schema and code kept in Git so any change can be traced.

4.4 System design

4.4.1 System architecture

The system has three layers. The device layer is the ESP32 with the ultrasonic sensor, the motion sensor and the RGB light. The network layer is the Wi-Fi connection between the board and the server, carrying small JSON messages. The application layer is the Flask application with the SQLite database, which holds the queue, the desk setting and the record of device readings. Figure 1 shows the three layers and the direction of the traffic between them.

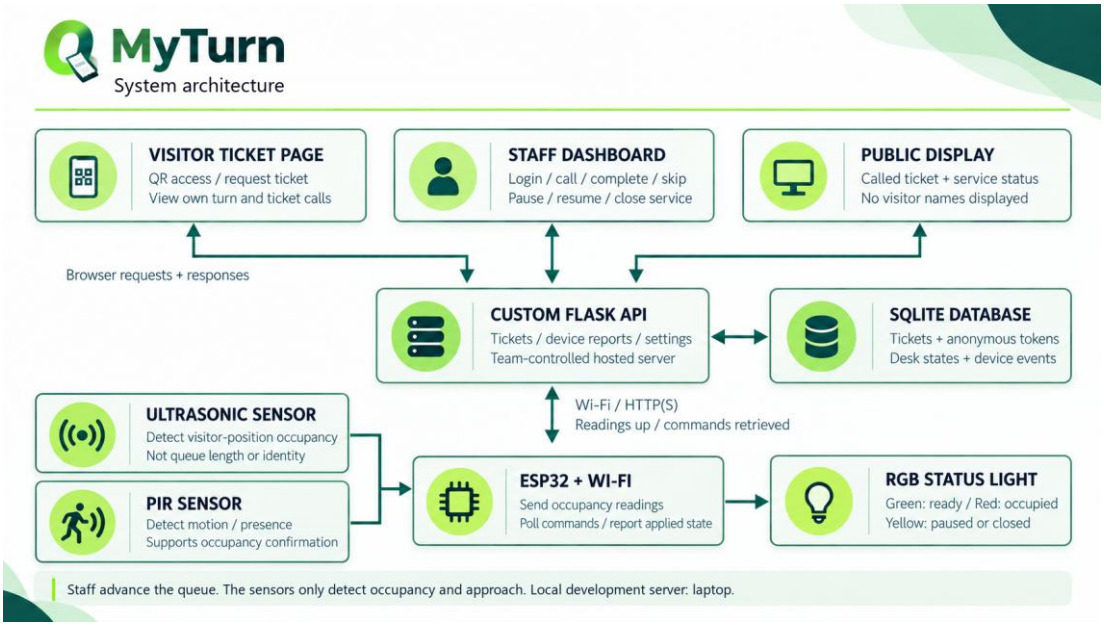


Figure 1: Architecture of the MyTurn system for one service desk

4.4.2 Use case diagram

Two people and one device use the system. A visitor takes a ticket and follows the queue from a phone. A staff member signs in, calls and closes tickets, and controls the state of the desk. The device does not act for a person: it reports what it measures and applies the setting the staff chose. Figure 2 shows the use cases and who may start each of them.

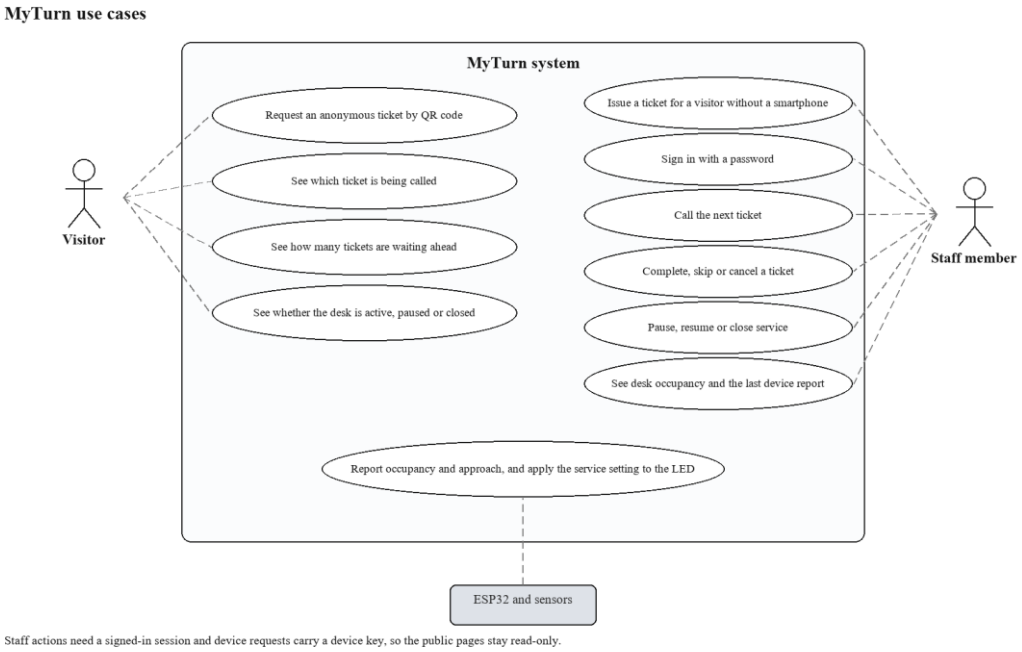


Figure 2: Use cases of the MyTurn system and the actors that start them

4.4.3 Data flow

Figure 3 follows one visit from the QR code to the status light. The visitor page and the staff dashboard never write to the database directly; they send a request to the API, which applies the rules in a transaction. The device posts a reading and asks for the current setting on a short interval, and the dashboard shows the state the device last reported rather than the state that was requested, so a command is never shown as applied while the board is offline.

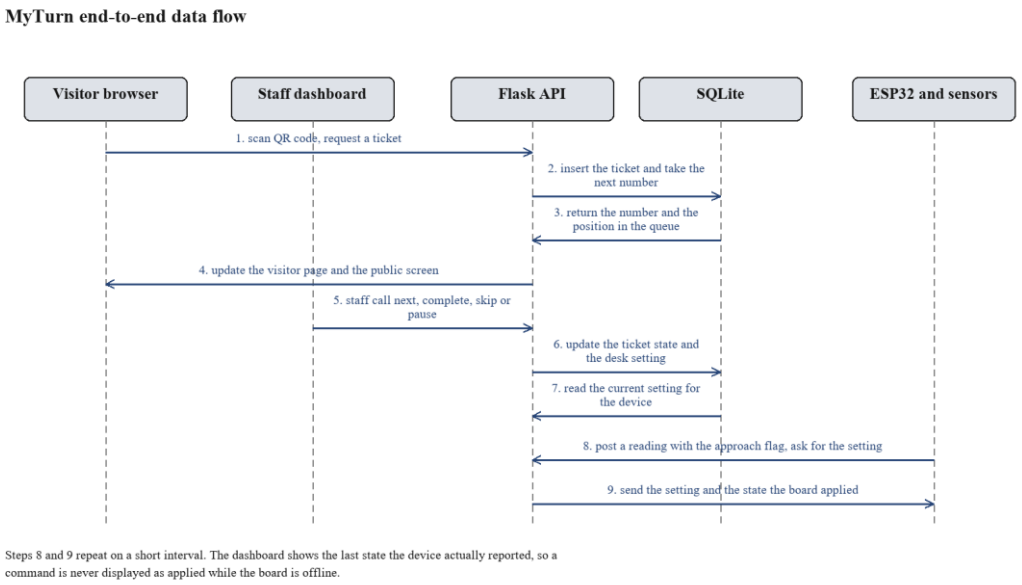


Figure 3: End-to-end data flow between the visitor, the staff, the API, the database and the device

4.4.4 Database design

The database is a single SQLite file with eleven tables. Six of them hold the queue, the desk state and the board: tickets, ticket_events, desk_settings, devices, device_reports and counters. desk_profile holds the name, description and target time the console edits, and four tables hold the staff accounts: users, invitations, password_resets and account_events. Figure 4 shows the main queue and device tables and their relations, and Table 7 describes all eleven. Ticket numbers are taken from a counter row inside the same transaction that inserts the ticket, so two visitors who request a ticket at the same moment cannot receive the same number.

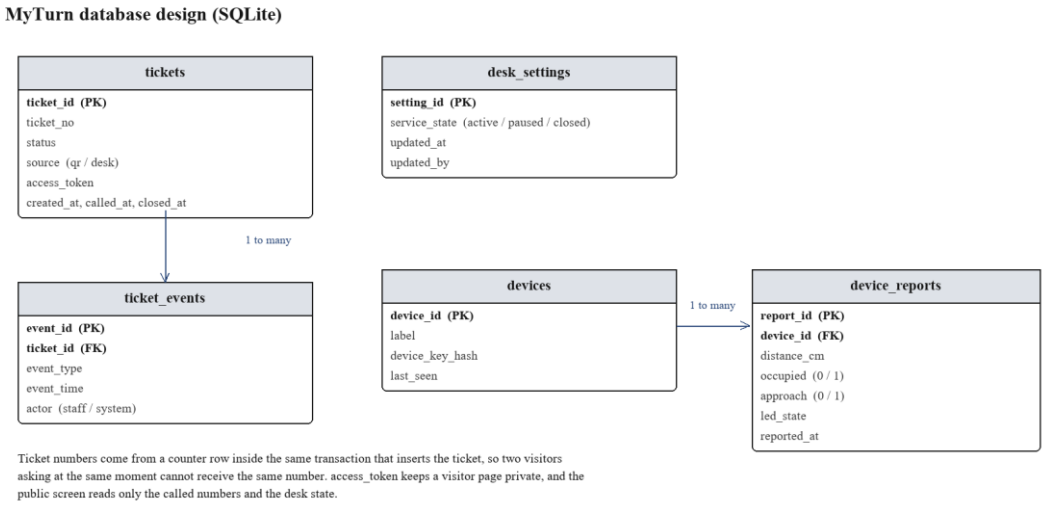


Figure 4: Entity relationship diagram of the MyTurn database

Table 7: Tables in the MyTurn database

Table	Key fields	Purpose
tickets	ticket_id, ticket_no, status, source, access_token, created_at, called_at, closed_at	One row per ticket, with the number, the current state and the times of the main events
ticket_events	event_id, ticket_id, event_type, detail, event_time, actor	A record of every change to a ticket, with the reason when the type alone does not say it, so the order and the cause can be checked afterwards
desk_settings	setting_id, service_state, updated_at, updated_by	The state the staff chose for the desk: active, paused or closed, appended with the author of each change
desk_profile	profile_id, name, description, target_seconds, updated_at, updated_by	The desk as it describes itself. The newest row is what the ticket page shows; older rows keep the previous wording and its author
devices	device_id, label, device_key_hash, last_seen	The registered device, the hash of its key and the time it last contacted the server
device_reports	report_id, device_id, distance_cm, occupied, approach, led_state, reported_at	The readings sent by the device, including the approach flag and the state the board applied
counters	name, value	The source of the next ticket number, updated inside the ticket transaction
users	user_id, full_name, email, password_hash, role, is_active, created_at, last_login_at	One row per staff member or administrator, with the role the server checks
invitations	invitation_id, email, role, token_hash, invited_by, created_at, expires_at, accepted_at	A single-use invitation. Only the digest of its token is stored
password_resets	reset_id, user_id, token_hash, created_at, expires_at, used_at	A single-use password-reset link, with its expiry and the moment it was used
account_events	event_id, event_type, email, detail, actor, event_time	What happened to an account, by address, so the history survives a rename or a deleted row

4.4.5 Hardware design

The circuit is small. The ESP32 drives the sensor trigger pin, reads the echo pin through a voltage divider, reads the digital output of the motion sensor and drives the three LED channels, each through its own resistor. Table 8 lists the connections and Figure 5 shows the arrangement. The divider is needed because the distance sensor is powered from 5 V and its echo output is therefore about 5 V, while the ESP32 input should not be taken above 3.3 V. The motion sensor runs from the same pin but switches at 3.3 V, so its output goes straight to the board and no divider is needed. The two sensors are mounted at different angles: the distance sensor looks down the service position, and the motion sensor looks across the approach to the desk.

Table 8: Pin connections between the ESP32, the sensors and the status light

ESP32 pin	Connected to	Note
GPIO 5	HC-SR04 Trig	Sends the short trigger pulse that starts a measurement
GPIO 18	HC-SR04 Echo through the 1 kΩ / 1.8 kΩ divider	Reads the pulse duration; the divider keeps the voltage near 3.2 V
5 V (VIN)	HC-SR04 VCC	Supplies the distance sensor from the USB input
GPIO 19	HC-SR501 OUT	Reads the motion signal; the module switches at 3.3 V, so it needs no divider
5 V (VIN)	HC-SR501 VCC	Supplies the motion module from the same USB input
GPIO 25	Red channel of the LED through 220 Ω	Lit together with green to show the paused or closed state
GPIO 26	Green channel of the LED through 220 Ω	Shows the active and free state
GPIO 27	Blue channel of the LED through 220 Ω	Wired and tested, kept for later use
GND	Sensor grounds, LED common cathode and divider ground	One common ground for the whole circuit

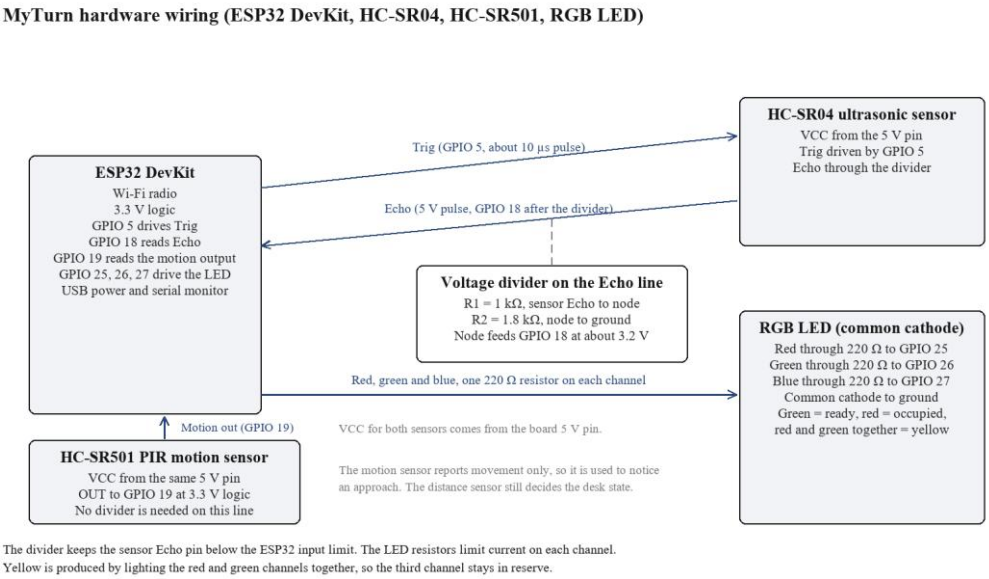


Figure 5: Wiring of the ESP32, the ultrasonic sensor, the motion sensor and the RGB status light

4.5 Implementation

4.5.1 Device firmware

The firmware on the ESP32 does five things in a loop. It triggers the distance sensor and measures the echo time, converts the time to centimetres, keeps a median of the last five measurements and decides whether the position is occupied. It reads the

motion sensor, and when that output goes high it starts an approach, takes distance readings more often for as long as the approach lasts, and closes the event as a walk-past if no occupancy follows within ten seconds. The motion signal is ignored for the first minute after power-up, because the module needs that time to settle. The board posts a reading, carrying the approach flag, to the API at a fixed interval. It asks the API for the current service setting and applies it to the LED, and it remembers the last answered setting, so a lost message does not leave the light in the wrong state. The sketch is listed in Appendix A. Credentials are kept in a small configuration header that is not committed to the repository.

Two details matter for reliability. The board sends the state it has applied with every reading, which lets the dashboard show what the light is doing rather than what it was asked to do. The board also retries a failed request twice before dropping the reading, so a short interruption of the Wi-Fi does not create a gap in the device history.

4.5.2 Application programming interface

The API is a single Flask application. Table 9 lists the routes. Public routes can be called by any browser, account routes require a signed-in session and some of them an administrator, and device routes require the device key sent in a request header. The queue rules live in queue_rules.py and are shared by the desk routes and the public state; the route functions validate the request and return the answer.

Table 9: API routes of the MyTurn application

Route	Called by	What it does
GET /api/services	Visitor page	Returns the service this desk offers, its description and whether it is taking tickets
POST /api/tickets	Visitor page	Creates a ticket and returns the number, the position in the queue and the access token
GET /api/tickets/<token>	Ticket page	Returns the number, the state of the ticket and how many tickets are ahead
GET /api/public/state	Public screen, ticket page	Returns the called number, the desk state, the waiting count and the measured service time
GET /api/events	Public screen, ticket page	Holds a Server-Sent Events stream open and sends the state again whenever the queue changes
POST /api/auth/login	Sign-in page	Checks an account and starts a session with the role and a CSRF token

Route	Called by	What it does
POST /api/auth/logout, GET /api/auth/me	Signed-in pages	Ends the session, or reports the account and token behind it
POST /api/auth/register, GET /api/auth/invitation	Invitation page	Reads an invitation and creates the staff account it allows
POST /api/auth/forgot-password, /api/auth/reset-password	Password pages	Issues a single-use reset link and changes the password from it
POST /api/auth/password	Account page	Changes the password of the signed-in account after checking the current one
GET /api/auth/users, POST /api/auth/invitations, POST /api/auth/invitations/<id>/revoke	Administration	Lists accounts and invitations, sends an invitation and withdraws one
POST /api/auth/users/<id>/active, /api/auth/users/<id>/role	Administration	Switches an account on or off, or moves it between staff and administrator
GET /api/auth/activity	Activity page	Merges ticket, desk and account history into one newest-first list
GET /api/auth/stats, GET /api/auth/tickets.csv	Statistics page	Counts and service times, or every ticket as a CSV file
GET, POST /api/auth/desk	Desk settings page	Reads or appends the desk name, description and target service time
POST /api/staff/login	Staff dashboard	Checks the shared classroom password and starts a session
POST /api/staff/logout	Staff dashboard	Ends the session
GET /api/staff/queue	Staff dashboard	Returns the waiting tickets, the desk state, the current call, the effective hold and the last device report
POST /api/staff/call-next	Staff dashboard	Calls the next waiting ticket inside one transaction, requeues an expired call first and refuses a second call while the hold is running
POST /api/staff/tickets	Staff dashboard	Issues a walk-in ticket from the desk
POST /api/staff/tickets/<id>/action, /recall, /undo	Staff dashboard	Completes, skips or cancels a ticket, announces it again, or takes back the last decision
POST /api/staff/service	Staff dashboard	Pauses, resumes or closes the desk and records who changed the setting
POST /api/device/readings	ESP32	Stores a distance reading, the occupancy state, the approach flag and the LED state the board applied
GET /api/device/setting	ESP32	Returns the current service setting for the light and updates the last seen time of the device

4.5.3 Browser interfaces

Fourteen pages were built. The home page explains the system and needs no login. The ticket page opens from the QR code; it shows the service, the waiting count, the measured wait and the state of the desk, and its one button issues a number. The

tracked ticket page repeats the number, the state and how many tickets are ahead, so a visitor can keep it open on their phone. The public screen is the same information in a large layout meant for a monitor or a tablet in the waiting area. Four account pages cover signing in, joining from an invitation, asking for a reset link and choosing a new password.

The staff dashboard carries the working end of the system: the queue, the number being called with its hold countdown, buttons to complete, skip or cancel, a walk-in ticket, recall, undo, a control to pause, resume or close the desk, and a line showing when the device last reported, whether a visitor is approaching and what state the board applied. The administration section has four pages: the team and its invitations, the activity history, the statistics with the CSV export, and the desk settings. The account page changes the password of the signed-in person. The ticket page and the public screen follow the queue over a Server-Sent Events stream, so a call arrives without a refresh; the console pages poll the API on intervals between two and thirty seconds, and the queue pages fall back to polling if the stream cannot be opened.

The layout keeps the visitor side to one screen on a phone. Numbers are large, colours match the light (green for active and free, red for occupied, yellow for paused or closed), and no page asks a visitor to type anything. Colour is never the only signal: each state also carries a word and a short sentence, so a visitor who cannot tell the colours apart can still read the state.

YOUR NUMBER

2

Waiting

You are in the queue

Stay nearby, or keep this page open. It updates on its own.

You are next in line.

NOW SERVING

1

At the desk now

PEOPLE WAITING

3

3 visitors

ESTIMATED WAIT

Not available

The desk has not served enough visitors yet to measure this

DESK STATUS

Open

Service: **General Service**

Position: **0 people ahead**

Last checked 18:54 Refresh

Remove this ticket from this phone

Figure 6: Visitor page on a phone, showing the ticket number, the called number and the desk state

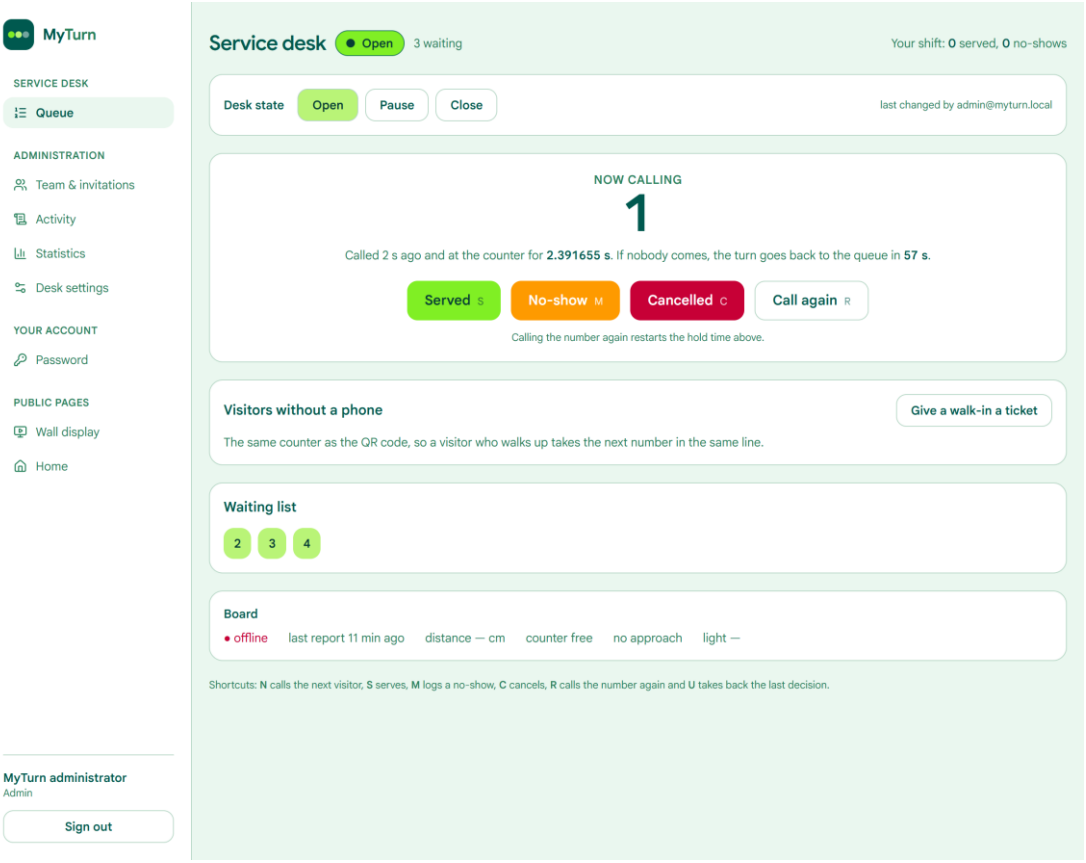


Figure 7: Staff dashboard, showing the queue and the call, complete and skip actions

[space for the image, about half a page]

Figure 8: The desk during a test, with the sensor above the service position and the status light in view

4.6 Testing and validation

The nine acceptance checks from Chapter Three were run on the last day of the build, after the fourth iteration was finished. Table 10 gives the check, the expected result, the result that was observed and the outcome. The highlighted cells are filled in from the test record sheet in Appendix D.

Table 10: Results of the acceptance tests

No.	Check	Expected result	Observed result	Outcome
1	Ticket order	Twenty numbers, unique and in order, including two tickets requested at the same moment	to be recorded	pass or fail
2	Occupancy detection	At least 18 of 20 labelled trials classified correctly	to be recorded	pass or fail
3	Approach detection	At least 9 of 10 approaches reported within 2 seconds	to be recorded	pass or fail

No.	Check	Expected result	Observed result	Outcome
4	Walk-past rejection	No occupancy reported in 10 walk-past trials	to be recorded	pass or fail
5	Call notification	Called number on the visitor page and the public screen within 5 seconds	to be recorded	pass or fail
6	Remote light control	Light yellow and then green within 5 seconds of each staff action	to be recorded	pass or fail
7	Persistence	Tickets, events and the desk setting still present after a restart	to be recorded	pass or fail
8	Connection loss	Board reconnects by itself after 30 seconds and the queue is unchanged	to be recorded	pass or fail
9	Access control	Staff request without a session and device request with a wrong key both refused	to be recorded	pass or fail

4.7 Results and discussion

4.7.1 Results

This section reports what the tests produced. The counts, the delays and the chart of the occupancy trials are taken from the test record sheet in Appendix D and from the rows the system stored in SQLite.

Occupancy. Out of twenty labelled trials the prototype classified <<number>> correctly, which is <<percentage>>. Of the trials where the position was occupied, <<number>> were recognised, and of the ten empty trials <<number>> were reported as empty. The trials that were classified incorrectly are described in the record sheet, together with the distance the sensor measured at the time.

[space for the image, about half a page]

Figure 9: Occupancy trial results: correct and incorrect classifications out of twenty labelled trials

Approach detection. The motion sensor reported <<number>> of the ten approaches within two seconds of the visitor entering the marked zone. In the ten walk-past trials the desk was reported as occupied <<number>> times. Because the module stops responding while a person stands still, it is never allowed to decide the occupancy state: the distance median keeps that decision, and the approach flag only raises the sampling rate and shows the short hint on the dashboard.

Order and timing. All <<number>> ticket numbers issued during the test were unique, and the order in which the tickets appear in the database matches the order in which they were requested, including the two requests sent at the same moment. A call reached both browser pages in <<number>> seconds on average, with a longest wait of <<number>> seconds, and a pause or resume reached the light in <<number>> seconds on average. The device reported every <<number>> seconds throughout the test, and the dashboard showed the applied LED state rather than the requested one at every check.

[space for the image, about half a page]

Figure 10: Delay between a staff action and the result showing on the page and on the light

Persistence and connection handling. After the server and the board were restarted, the system still held <<number>> tickets, <<number>> ticket events and the desk setting that had been chosen before the restart. When the access point was switched off for thirty seconds, the board reconnected after <<number>> seconds without help, and no ticket was lost or reordered during the interruption.

4.7.2 Discussion of the results

The results show that the three questions behind the project can be answered with the parts that were bought. The order of the queue no longer depends on memory or on being physically present, because every ticket is written to the database before it is shown and the audit table keeps the sequence of events. The state of the desk no longer depends on a visitor looking through a doorway, because the light is driven by a setting the staff control and by a sensor that watches one position. The device receives commands without the server needing to reach into the campus network, because the board asks for the setting on an outbound connection.

The measurements also show where the design is weak. The occupancy results are the least reliable part of the system, and the trials that failed came from the known behaviour of an ultrasonic sensor rather than from a mistake in the circuit: a person who stands almost still returns a weak echo, and a person who leans at an angle sends the reflection away from the module. Requiring five consistent readings before the state may change reduces false changes, and it also delays the change by up to two seconds, which is a deliberate trade between stability and speed. A different sensor would reduce the problem at a higher price.

The second sensor earned its place, but for a limited job. The approach signal turns a walk past the counter into an event that can be counted, instead of a reading that might be mistaken for a visitor, and it shortens the time between a visitor arriving and the occupied state being confirmed, because the board samples the distance more often while an approach is in progress. It does not make the occupancy decision more accurate, and it brings its own limitation: the module is blind for about a minute after power is applied, which is why its output is ignored during that time rather than trusted.

The polling design has a cost that the tests made visible. The board asks for the setting instead of receiving it, so a command waits for the next poll before it is applied, which is why the delay is measured in seconds. The same interval keeps the request rate low and works through the network the board already uses, which matters more for one desk than the last second of latency.

Two decisions changed during the build because of what testing showed, and both are recorded in Table 3. Calling the next ticket had to be moved into a single database transaction after two staff windows were found to call the same ticket, which is the kind of fault that appears only when two clients act together. The handling of a lost network connection had to be improved after the board was found to keep an old setting, which is why the reading now carries the state the board applied and the dashboard displays it.

The results should be read within the limits set in Chapter One. The trials were run in a laboratory with team members acting as visitors, in one session, with one board and two sensors. The figures describe this prototype under those conditions. A long deployment at a real desk, with visitors who move in their own way and with a Wi-Fi network shared with other users, would produce different numbers, and the design allows for it: the threshold, the hold time, the warm-up and the polling interval are all settings rather than constants in the code.

CHAPTER V: CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter closes the report. It states what the project achieved against its objectives, describes the difficulties met while building and testing the prototype, and lists the improvements that would make the system more useful at a real service desk.

5.2 Conclusion

The project set out to design, implement and demonstrate a low-cost IoT prototype that organises visitors at one service desk and reports ticket calls, desk occupancy and staff-controlled service status through a web application and a physical indicator. The prototype exists and each of the five specific objectives was met. Numbered tickets are issued in order and reach the database before they are shown, and staff can issue a ticket for a visitor who has no smartphone. The ESP32 with the ultrasonic and motion sensors detects whether someone is standing at the service position, notices a visitor walking towards the counter, and records each state with a timestamp. The Flask API and the SQLite database carry the readings, the queue and the control path without any use of Blynk, ThingSpeak or a similar platform. The visitor page, the public screen and the staff dashboard give each group the part of the information it needs, and the acceptance tests were run and recorded with the results in Chapter Four.

Taken together, the parts answer the question that started the work. A visitor who is standing in the corridor can now see which ticket is being served, how many tickets are waiting and whether the desk is active, paused or closed, and a visitor who steps away keeps a place that is written down. Staff control the state of the desk from one page, and the light shows that state to anyone who passes. The whole system runs on about RWF 46,200 of hardware and free software, and the queue data stays on a server the team controls.

Three limits should be kept in view when the results are read. The evaluation used twenty labelled trials in a laboratory, so the accuracy figures describe the prototype under test conditions and not a long deployment. The prototype serves one desk, and nothing in this report shows how the design would behave with several counters working together. The queue stores no personal data by design, which also means a

visitor who loses the page cannot be found again by name. Within those limits, the work shows that the uncertainty a visitor feels at a service desk can be reduced with parts a student team can buy and code.

5.3 Challenges encountered

Time was the tightest constraint. Four days is enough for one working part per day, and it is not enough for integration to be left until the end. The plan was kept by agreeing the minimum feature set before coding began and by refusing to add features outside it, which meant that some ideas, such as a waiting time estimate, were written down and postponed.

The sensor caused most of the debugging. The first threshold worked when a hand was moved in front of the module and failed when a person stood still, because the echo from soft clothing is weak and a still body gives inconsistent returns between pulses. The module also answers the first reflection it hears, so a nearby wall produced readings that had nothing to do with the visitor. The answer was a fixed mounting position, a median of five readings and a two second hold before the state may change, and the price is a small delay when someone arrives or leaves. The motion sensor brought a smaller version of the same lesson. It needs about a minute after power-up before its output can be trusted, and it says nothing at all about a visitor who is standing still, so it was given the narrow job of noticing an approach while the distance sensor kept the decision.

The network shaped the control path. The board sits behind a router, so the server cannot open a connection to it, and an early attempt to have the dashboard send the command directly did not work. Turning the control path around so that the board asks for the setting solved the problem and left the dashboard with the second problem of showing what the board had actually applied. A short break in the Wi-Fi also left the board repeating an old setting until the reading was made to carry the applied state.

The queue held a fault that the early tests missed. Two staff windows calling the next ticket at the same moment produced two calls for one ticket, which the first tests did not reveal because only one window was open. Moving the choice into a single transaction, and refusing a call while another ticket is being served, removed the fault.

Hosting was the last practical difficulty. A student budget does not include a server, so the application was prepared to run either on a small free host or from a laptop through a tunnel, and the choices were written into the configuration rather than the code. Serving the pages over HTTPS mattered because the phone camera needs it to open the QR code link, and persuading the team that the insecure default was not acceptable took longer than the change itself.

5.4 Recommendations and future work

A better sensor would help most. A millimetre-wave radar or a time-of-flight module would hold a still person more reliably than an ultrasonic module, and the code already keeps the detection settings in one place, so the change would be mostly electrical. The motion sensor that was added watches the approach and nothing else; a second one at the waiting area, or a small counting sensor at the door, would let the system check that the number of waiting tickets matches the number of people in the room.

Communication is the next limit. Polling keeps the design simple for one device, and it will not scale to several desks. Moving the device traffic to MQTT with a broker would let the server push a command and would cut the delay to a fraction of a second, at the cost of running the broker and keeping it available.

The prototype could be extended to more than one desk. The database already keeps a device table and a desk setting, and a desk column on the ticket table, together with a rule for transferring a ticket, would be the next step. The public screen would then need to show which counter is calling, and the visitor page would need to say where to go.

The history the system already stores could be used. Every ticket event carries a timestamp, so the average service time can be calculated from the last few tickets and shown as an estimated wait. This was left out of the prototype on purpose, because a wrong estimate is worse than no estimate, and the feature needs a week of real data before it can be trusted.

Reach is the remaining gap. A visitor who leaves the building cannot see the screen, so a message by SMS or WhatsApp would be useful once the system has a budget for

the sending cost. A small display at the door, lit by the same board, would cover the same gap without a telephone number.

The evaluation should be repeated over a longer period. The tests in this report were run in one session with twenty trials. Running the prototype at a real desk for two weeks would show whether the classification rate holds when visitors move in their own way, whether the notification delay is short enough at busy times, and whether staff keep using the call button once the novelty has passed. The same period would provide the data needed for the waiting time estimate.

Finally, accessibility and security deserve attention before any real use. The pages should be readable in Kinyarwanda as well as English, the called number should be announced with a sound for visitors who cannot see the screen, and the staff login should be reviewed with password hashing, a limit on failed attempts and a log of the actions taken from the dashboard.

REFERENCES

Adafruit. (n.d.). PIR passive infrared proximity motion sensor. https://learn.adafruit.com/pir-passive-infrared-proximity-motion-sensor

Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. Computer Networks, 54(15), 2787–2805.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile software development. https://agilemanifesto.org

Bitner, M. J. (1990). Evaluating service encounters: The effects of physical surroundings and employee responses. Journal of Marketing, 54(2), 69–82.

Blynk. (2023). Blynk documentation. https://docs.blynk.io

Dargie, W., & Poellabauer, C. (2010). Fundamentals of wireless sensor networks: Theory and practice. John Wiley & Sons.

Elec Freaks. (n.d.). Ultrasonic ranging module HC-SR04 [Datasheet]. https://www.elecfreaks.com

Espressif Systems. (2023). ESP32 series datasheet (Version 4.6). Espressif Systems.

Fielding, R., & Reschke, J. (2014). Hypertext Transfer Protocol (HTTP/1.1): Message syntax and routing (RFC 7230). Internet Engineering Task Force.

Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. Future Generation Computer Systems, 29(7), 1645–1660.

Hipp, D. R. (2020). SQLite. https://www.sqlite.org

INES-Ruhengeri. (2025). Guidelines for dissertation and thesis writing. Institut d’Enseignement Supérieur de Ruhengeri.

Maister, D. H. (1985). The psychology of waiting lines. In J. A. Czepiel, M. R. Solomon, & C. F. Surprenant (Eds.), The service encounter (pp. 113–123). Lexington Books.

MathWorks. (n.d.). ThingSpeak: IoT analytics platform. https://thingspeak.com

OASIS. (2019). MQTT version 5.0 (OASIS Standard). OASIS Open.

Pallets Projects. (2023). Flask documentation (Version 2.3). https://flask.palletsprojects.com

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. Scrum.org.

Shelby, Z., Hartke, K., & Bormann, C. (2014). The Constrained Application Protocol (CoAP) (RFC 7252). Internet Engineering Task Force.

APPENDICES

Appendix A: ESP32 device firmware

The sketch below runs on the ESP32. It measures the distance, keeps a median, decides the occupancy state with a hold time, posts the reading with the state it applied, and asks the server for the current service setting. The credentials are kept in a separate configuration header that is not shared.

```
// MyTurn device: ESP32 with an ultrasonic and a PIR sensor, and an RGB light
#include <WiFi.h>
#include <HTTPClient.h>
#include "secrets.h"      // WIFI_SSID, WIFI_PASS, API_BASE, DEVICE_KEY

const int TRIG_PIN = 5, ECHO_PIN = 18, PIR_PIN = 19;
const int RED_PIN = 25, GREEN_PIN = 26, BLUE_PIN = 27;

const float OCCUPIED_CM = 90.0;   // closer than this counts as a visitor
const int   SAMPLES     = 5;      // readings kept for the median
const int   HOLD_MS     = 2000;   // a new state must last this long
const int   INTERVAL_MS = 5000;   // how often the board talks to the server
const int   PIR_WARMUP_MS = 60000;   // motion is ignored this long after boot
const int   FAST_MS       = 1000;    // interval while an approach is in progress
const int   APPROACH_MS   = 10000;   // approach with no occupancy = walk-past

float window[SAMPLES];
int   nextSlot = 0;
bool  occupied = false, candidate = false, approaching = false;
unsigned long candidateSince = 0, lastSend = 0;
unsigned long approachSince = 0, bootMs = 0;
String applied = "yellow";

float measureCm() {
  digitalWrite(TRIG_PIN, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  unsigned long us = pulseIn(ECHO_PIN, HIGH, 30000);
  return us == 0 ? -1.0 : us / 58.0;   // microseconds to centimetres
}

float medianCm() {
  float sorted[SAMPLES];
  for (int i = 0; i < SAMPLES; i++) sorted[i] = window[i];
  for (int i = 1; i < SAMPLES; i++) {
    float value = sorted[i]; int j = i - 1;
    while (j >= 0 && sorted[j] > value) { sorted[j + 1] = sorted[j]; j--; }
    sorted[j + 1] = value;
  }
  return sorted[SAMPLES / 2];
}

void setLight(const String &state) {
  bool green = (state == "green" || state == "yellow");
  bool red   = (state == "red"   || state == "yellow");
  digitalWrite(GREEN_PIN, green ? HIGH : LOW);
  digitalWrite(RED_PIN,   red   ? HIGH : LOW);
  digitalWrite(BLUE_PIN,  LOW);
  applied = state;
```

```
}

bool sendJson(const String &path, const String &body, String &reply) {
  HTTPClient http;
  http.begin(String(API_BASE) + path);
  http.addHeader("Content-Type", "application/json");
  http.addHeader("X-Device-Key", DEVICE_KEY);
  int code = body.length() ? http.POST(body) : http.GET();
  bool ok = (code >= 200 && code < 300);
  if (ok) reply = http.getString();
  http.end();
  return ok;
}

void report(float cm, bool busy, bool near, String &setting) {
  String body = "{\"distance_cm\":" + String(cm, 1) +
                ",\"occupied\":" + String(busy ? 1 : 0) +
                ",\"approach\":" + String(near ? 1 : 0) +
                ",\"led_state\":\"" + applied + "\"}";
  String reply;
  for (int attempt = 0; attempt < 3; attempt++) {
    if (sendJson("/api/device/readings", body, reply)) break;
    delay(400);
  }
  if (sendJson("/api/device/setting", "", reply)) {
    setting = reply.substring(reply.indexOf(":") + 2, reply.length() - 2);
  }
}

void setup() {
  pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
  pinMode(PIR_PIN, INPUT);
  pinMode(RED_PIN, OUTPUT);  pinMode(GREEN_PIN, OUTPUT);
  pinMode(BLUE_PIN, OUTPUT);
  bootMs = millis();
  setLight("yellow");
  Serial.begin(115200);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  for (int i = 0; i < SAMPLES; i++) window[i] = 300;
}

void loop() {
  float cm = measureCm();
  if (cm > 0) { window[nextSlot] = cm; nextSlot = (nextSlot + 1) % SAMPLES; }
  float value = medianCm();
  bool busy = (value > 0 && value < OCCUPIED_CM);
  if (busy != occupied) {
    if (busy != candidate) { candidate = busy; candidateSince = millis(); }
    else if (millis() - candidateSince > HOLD_MS) { occupied = busy; }
  } else { candidate = occupied; }
  bool pir = digitalRead(PIR_PIN) == HIGH;      // true while motion is seen
  bool settled = (millis() - bootMs) > PIR_WARMUP_MS;
  if (settled && pir && !approaching) {
    approaching = true; approachSince = millis();
  }
  if (approaching && (occupied || millis() - approachSince > APPROACH_MS)) {
    approaching = false;              // occupied, or a walk-past
  }
  unsigned long interval = approaching ? FAST_MS : INTERVAL_MS;
  if (millis() - lastSend > interval) {
    String setting = "paused";
    report(value, occupied, settled && (approaching || pir), setting);
    String wanted = (setting == "active") ? (occupied ? "red" : "green") : "yellow";
    if (wanted != applied) setLight(wanted);
    lastSend = millis();
  }
```

```
}
```

Appendix B: Flask application routes

This extract shows the parts of the application that carry the rules: ticket creation inside one transaction, the call of the next ticket, and the two device routes. The full application also serves the three browser pages.

```
# MyTurn API (extract): tickets, staff actions and device traffic
from functools import wraps
from flask import Flask, g, jsonify, request, session
from datetime import datetime, timezone
import hashlib, os, secrets, sqlite3

app = Flask(__name__)
app.secret_key = os.environ["MYTURN_SECRET"]
DB = os.environ.get("MYTURN_DB", "myturn.db")

def db():
    if "db" not in g:
        g.db = sqlite3.connect(DB, isolation_level=None)   # transactions by hand
        g.db.row_factory = sqlite3.Row
    return g.db

@app.teardown_appcontext
def close_db(exc):
    conn = g.pop("db", None)
    if conn is not None:
        conn.close()

def now():
    return datetime.now(timezone.utc).isoformat(timespec="seconds")

def staff_required(view):
    @wraps(view)
    def wrapper(*args, **kwargs):
        if not session.get("staff"):
            return jsonify(error="sign in required"), 401
        return view(*args, **kwargs)
    return wrapper

def device_required(view):
    @wraps(view)
    def wrapper(*args, **kwargs):
        digest = hashlib.sha256(request.headers.get("X-Device-Key", "").encode())
        row = db().execute("select device_id from devices where device_key_hash = ?",
                           (digest.hexdigest(),)).fetchone()
        if row is None:
            return jsonify(error="unknown device"), 401
        g.device_id = row["device_id"]
        return view(*args, **kwargs)
    return wrapper

@app.post("/api/tickets")
def create_ticket():
    conn = db()
    conn.execute("begin immediate")            # one writer at a time
    try:
        seq = conn.execute("select value from counters where name = ?",
                           ("ticket",)).fetchone()["value"] + 1
        conn.execute("update counters set value = ? where name = ?", (seq, "ticket"))
        token = secrets.token_urlsafe(16)
        cur = conn.execute(
```

```
            "insert into tickets (ticket_no, status, source, access_token, "
            "created_at) values (?, ?, ?, ?, ?)",
            (seq, "waiting", request.get_json().get("source", "qr"),
             token, now()))
        conn.execute("insert into ticket_events (ticket_id, event_type, event_time, "
                     "actor) values (?, ?, ?, ?)", (cur.lastrowid, "created", now(),
                                                    "system"))
        conn.execute("commit")
    except Exception:
        conn.execute("rollback")
        raise
    ahead = conn.execute("select count(*) as c from tickets "
                         "where status = ? and ticket_no < ?",
                         ("waiting", seq)).fetchone()["c"]
    return jsonify(ticket_no=seq, ahead=ahead, token=token)

@app.post("/api/staff/call-next")
@staff_required
def call_next():
    conn = db()
    conn.execute("begin immediate")
    try:
        if conn.execute("select 1 from tickets where status = ?",
                        ("called",)).fetchone():
            conn.execute("commit")
            return jsonify(error="a ticket is already being called"), 409
        row = conn.execute("select ticket_id, ticket_no from tickets "
                           "where status = ? order by ticket_no limit 1",
                           ("waiting",)).fetchone()
        if row is None:
            conn.execute("commit")
            return jsonify(error="no ticket is waiting"), 404
        conn.execute("update tickets set status = ?, called_at = ? "
                     "where ticket_id = ?",
                     ("called", now(), row["ticket_id"]))
        conn.execute("insert into ticket_events (ticket_id, event_type, event_time, "
                     "actor) values (?, ?, ?, ?)",
                     (row["ticket_id"], "called", now(), session["staff"]))
        conn.execute("commit")
    except Exception:
        conn.execute("rollback")
        raise
    return jsonify(ticket_no=row["ticket_no"])

@app.post("/api/device/readings")
@device_required
def reading():
    data = request.get_json(force=True)
    conn = db()
    conn.execute("insert into device_reports (device_id, distance_cm, occupied, "
                 "approach, led_state, reported_at) values (?, ?, ?, ?, ?, ?)",
                 (g.device_id, data["distance_cm"], int(data["occupied"]),
                  int(data.get("approach", 0)), data["led_state"], now()))
    conn.execute("update devices set last_seen = ? where device_id = ?",
                 (now(), g.device_id))
    return jsonify(ok=True)

@app.get("/api/device/setting")
@device_required
def device_setting():
    row = db().execute("select service_state from desk_settings "
                       "order by setting_id desc limit 1").fetchone()
    return jsonify(service_state=row["service_state"] if row else "paused")
```

Appendix C: Database schema

The schema is created by this script when the server starts with an empty database.

```
-- MyTurn database schema (SQLite)
create table if not exists counters (
    name  text primary key,
    value integer not null default 0
);
insert or ignore into counters (name, value) values ('ticket', 0);

create table if not exists tickets (
    ticket_id    integer primary key autoincrement,
    ticket_no    integer not null unique,
    status       text    not null default 'waiting',
    source       text    not null default 'qr',
    access_token text    not null unique,
    created_at   text    not null,
    called_at    text,
    closed_at    text
);
create index if not exists idx_tickets_status on tickets (status, ticket_no);

create table if not exists ticket_events (
    event_id   integer primary key autoincrement,
    ticket_id  integer not null references tickets (ticket_id),
    event_type text    not null,
    event_time text    not null,
    actor      text    not null default 'system'
);

create table if not exists desk_settings (
    setting_id    integer primary key autoincrement,
    service_state text    not null,
    updated_at    text    not null,
    updated_by    text
);
insert into desk_settings (service_state, updated_at)
values ('paused', '2026-10-01');

create table if not exists devices (
    device_id       integer primary key autoincrement,
    label           text    not null,
    device_key_hash text    not null unique,
    last_seen       text
);

create table if not exists device_reports (
    report_id   integer primary key autoincrement,
    device_id   integer not null references devices (device_id),
    distance_cm real,
    occupied    integer not null default 0,
    approach    integer not null default 0,
    led_state   text,
    reported_at text    not null
);
```

Appendix D: Test record sheet

The sheets below were used during the acceptance tests. The occupancy sheet and the approach sheet were completed during the trials, and the timing sheet was completed from the timestamps stored in the database.

D.1 Occupancy trials

Table 11: Occupancy trial record sheet

Trial	Label	Distance read (cm)	Reported state	Correct (yes / no)
1	occupied			
2	occupied			
3	occupied			
4	occupied			
5	occupied			
6	occupied			
7	occupied			
8	occupied			
9	occupied			
10	occupied			
11	empty			
12	empty			
13	empty			
14	empty			
15	empty			
16	empty			
17	empty			
18	empty			
19	empty			
20	empty			

D.2 Approach and walk-past trials

Table 12: Approach and walk-past record sheet

Trial	Type	Approach reported (s)	Desk state shown	Correct (yes / no)
1	approach			
2	approach			
3	approach			

Trial	Type	Approach reported (s)	Desk state shown	Correct (yes / no)
4	approach			
5	approach			
6	approach			
7	approach			
8	approach			
9	approach			
10	approach			
11	walk-past			
12	walk-past			
13	walk-past			
14	walk-past			
15	walk-past			
16	walk-past			
17	walk-past			
18	walk-past			
19	walk-past			
20	walk-past			

D.3 Timing of notifications and light control

Table 13: Timing record sheet

Action	Time sent	Time shown on the page	Time shown on the light	Delay (s)
Call next ticket				
Pause the desk				
Resume the desk				
Close the desk				

D.4 Ticket order

Table 14: Ticket order record sheet

Request order	Ticket number issued	Note
1		
2		
3		
4		
5		

Request order	Ticket number issued	Note
6		
7		requested at the same moment as ticket 6
8		requested at the same moment as ticket 5
9		
10		
11		
12		
13		
14		
15		
16		
17		
18		
19		
20		